101: Java

May, 2023

Objective

- ~~インストール: Intellij IDEA, JDK~~
- Hello world!
- 文法
- Data types
- Operators (演算子)
- Loops (繰り返し処理)
- Condition (条件分岐)
- Array (配列)
- Exception (例外)
- Files and I/O
- Scanner

JDK

- Java Development Kit 17
  - https://www.oracle.com/java/technologies/downloads/#java17
  - Windows: x64 Installer or x64 MSI Installer
  - Debian/Ubuntu: `sudo apt update && sudo apt install -y openjdk-17-jdk`
- 環境変数?
  - `java -version`
  - `javac -version`

~~Intellij IDEA~~

- ~~Intellij IDEA Community Edition~~
  - ~~https://www.jetbrains.com/idea/download/~~

MJBK防止策: UTF-8

For example, some Shift-JIS characters include a backslash (0x5C "\") in the second byte, which is used as an escape character in many programming languages.

構		わ		な		い	
8d	**5c**	82	ed	82	c8	82	a2

A parser lacking support for Shift JIS will recognize 0x5C 0x82 as an invalid escape sequence, and remove it.[3] Therefore, the phrase cause mojibake.

高		塢		ネ	い	
8d		82	ed	82	c8	82	a2

https://en.wikipedia.org/wiki/Japanese_language_and_computers

Hello world

```
public class MyFirstJavaProgram {              //class

    /* First java program.
     * This will print 'Hello world' as the output
     */

    public static void main(String[] args) {       //main() method not Main
        System.out.println("Hello world");     /*prints "Hello world"*/
    }
}
```

```
javac MyFirstJavaProgram.java
java MyFirstJavaProgram
```

```
#include <stdio.h>
int main() {
   printf("Hello, world!\n");
   return 0;
}
```

```
#include <iostream>
int main() {
    std::cout << "Hello world!\n";
    return 0;
}
```

```
print("Hello world")
```

Object-oriented programming (OOP)

- Objects

States(属性/状態)

♂/♀、年齢、白柴/胡麻柴/黒柴, etc.

Behaviors(操作/動作)

- Class
  - blueprint/template (e.g. DNA, RNA)

例: class Dog

```
public class Dog {
  String breed;
  int age;
  String color;

  void bark() {
    System.out.println("Wan wan");
  }

  void eat() {
  }

  void sleep() {
  }
}
```

Keywords: abstract, boolean, break, byte, case, catch, char, class, do, for, if, …

Constructors

MyClass.java

```
public class MyClass {
  int num;
  MyClass() {
    num = 100;
  }
}
```

constructor

ConsDemo.java

```
public class ConsDemo {
  public static void main(String[] args) {
    MyClass t1 = new MyClass();
    MyClass t2 = new MyClass();
    System.out.println(t1.num + " " + t2.num);
  }
}
```

new objects

Constructors

MyClass2.java

```
class MyClass2 {
  int x;

  MyClass2(int i) {
    x = i;
  }
}
```

instance variable

constructor

parameter

ConsDemo2.java

```
public class ConsDemo2 {
  public static void main(String[] args) {
    MyClass2 t1 = new MyClass2(10);
    MyClass2 t2 = new MyClass2(20);
    System.out.println(t1.x + " " + t2.x);
  }
}
```

- a special method, matches the class name
- does not have a return type (void, int, etc.)
- is called when the object is created
- All Java classes have constructors

```java
public class Puppy {
  int puppyAge;                        //instance variable, outside any method

  Puppy(String name) {
    // This constructor* has one parameter, name.
    System.out.println("The name is: " + name);
  }

  public void setAge(int age) {          //This method takes one parameter, age.
    puppyAge = age;
  }

  public int getAge() {
    System.out.println("Puppy's age is: " + puppyAge);
    return puppyAge;
  }

  public static void main(String[] args) {
    // create an object myPuppy
    Puppy myPuppy = new Puppy("Cody");

    myPuppy.setAge(2);

    myPuppy.getAge();

      /* read age again */
    System.out.println("Variable Value: " + myPuppy.puppyAge);
  }
}
```

Primitive data types

- byte (8 bit signed integer)
  - byte a = 100;
  - min: -128 (-2^7), max: 127 (2^7-1)
- short (16 bit signed integer)
  - short b = 11000;
  - min: -32,768 (-2^15), max: 32,767 (2^15-1)
- int (32 bit signed integer)
  - int c = -2000000;
  - min: -2^31, max: 2^31-1
- long (64 bit signed integer)
  - min: -2^63, max: 2^63-1

default: 0

- float (32 bit single precision floating point)
  - float f = 22.22;
- double (64 bit double precision floating point)
  - double d = 33.4444;
- boolean
  - boolean isAlive = true;
  - boolean isDead = false;

default: false

- char (single 16 bit Unicode character)
  - char letterA = 'A';

Reference data types

```
Puppy myPuppy = new Puppy("Cody");
```

reference variable

default: null

Variable types

```
public class Test {
  public void puppyAge() {
    int age = 0;
    age = age + 7;
    System.out.println("Puppy age is: " + age);
  }

  public static void main(String[] args) {
    Test test = new Test();
    test.puppyAge();
  }
}
```

```
public class Test {
  public void puppyAge() {
    int age;          //エラー、初期化されてない
    age = age + 7;
    System.out.println("Puppy age is: " + age);
  }

  public static void main(String[] args) {
    Test test = new Test();
    test.puppyAge();
  }
}
```

Local variables: declared in methods, constructors

“変数宣言”

no modifiers, no default value

```java
import java.io.*;
public class Employee {

  // this instance variable is visible for any child class.
  public String name;

  // salary variable is visible in Employee class only.
  private double salary;

  // The name variable is assigned in the constructor.
  public Employee (String empName) {
    name = empName;
  }

  // The salary variable is assigned a value.
  public void setSalary(double empSal) {
    salary = empSal;
  }

  // This method prints the employee details.
  public void printEmp() {
    System.out.println("name: " + name);
    System.out.println("salary:" + salary);
  }

  public static void main(String[] args) {
    Employee empOne = new Employee("Rajesh");
    empOne.setSalary(100000);
    empOne.printEmp();
  }
}
```

instance variable

declared outside any method or constructor, inside a class
“public””private”: modifiers
in a static method, should be called using fully qualified name.
empOne.name

no nested method

```java
import java.io.*;
public class Employee {

  // salary variable is a private static variable
  private static double salary;

  // DEPARTMENT is a constant
  public static final String DEPARTMENT = "システム開発部";

  public static void main(String[] args) {
    salary = 300000;
    System.out.println(DEPARTMENT + " average salary: " + salary);
  }
}
```

class/static variables

from outside class:
Employee.DEPARTMENT

Modifiers

- Access modifiers
  - (空欄)
  - public
  - private
  - protected
- Non access modifiers
  - static (method, class)
  - final
  - abstract
  - synchronized*, volatile*

```
public class Dog {
  private String birthday;

  public String getBirthday() {
    return this.birthday;
  }

  public void setBirthday(String birthday) {
    this.birthday = birthday;
  }
}
```

Operators

- +
- -
- *
- /
- %
- ++
- --

- ==
- !=
- >
- <
- >=
- <=
- &&
- ||
- !
- =

Loops

```
public class TestLoop1 {

  public static void main(String[] args) {
    int x = 10;

    while(x < 20) {              // also try x <= 20
      System.out.print("value of x: " + x);
      x++;
      System.out.print("\n");          //手動で改行 escape sequence
    }
  }
}
```

Loops

```
public class TestLoop2 {

  public static void main(String[] args) {

    for(int x = 10; x < 20; x = x + 1) {
      System.out.println("value of x: " + x );
    }
  }
}
```

Loops

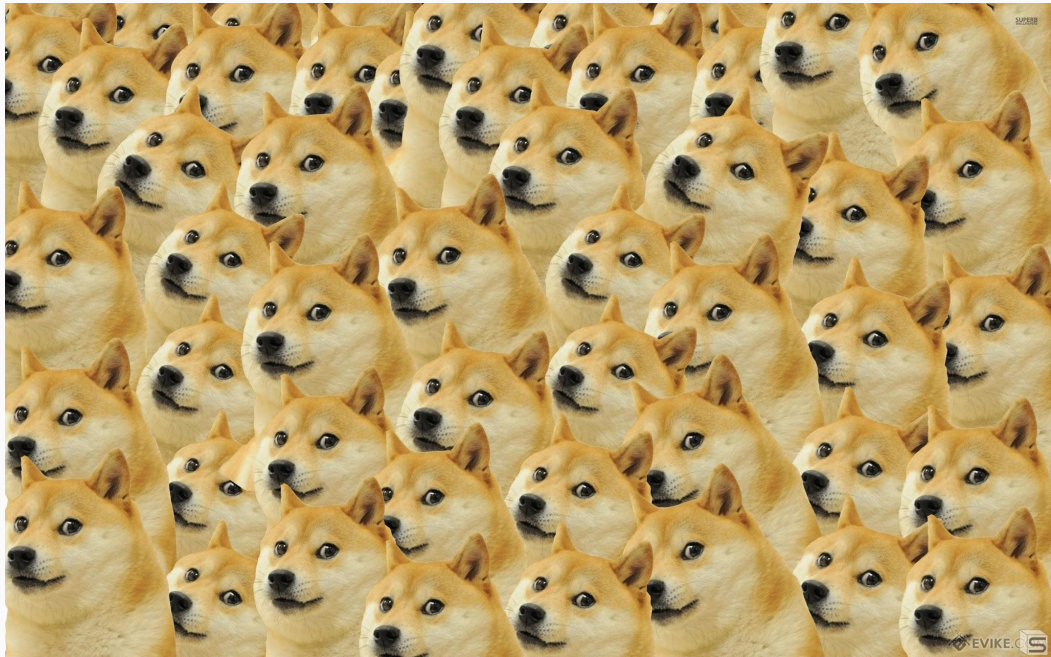
```
public class TestLoop3 {

    public static void main(String[] args) {
        int x = 10;

        do {
            System.out.println("value of x: " + x );
            x++;
        } while(x < 20);
    }
}
```

```
public class TestLoop4 {

    public static void main(String[] args) {
        int i = 8;
        while (true) {
            System.out.println(i);
            i++;
            if (i == 20) {
                System.out.
println("i is 20");
                break;
            }
        }
    }
}
```



```java
public class TestLoop5 {

    public static void main(String[] args) {
        int x = 8;
        for (int i=1; i<10; i++) {
            x++;
            if (x % 3 == 0) {
                continue;
            }
            System.out.println(x + " x%3!=0");
        }
        System.out.println("Value of x: " + x);
    }
}
```

```java
public class TestLoop6 {

    public static void main(String[] args) {
        int x = 8;
        for (int i=1; i<10; i++) {
            num++;
            if (x % 2 == 0) {
                System.out.println("even number: " + x);
            } else {
                System.out.println("odd number: " + x);
            }
        }
    }
}
```